中南大学考试试卷（答案）

2012--2013　学年 2 学期　　时间 100 分钟　2013 年 6 月 4 日

嵌入式系统　课程　32　学时 2.0 学分　考试形式：　开　卷

专业年级：　测控 10 级　　总分 100 分，占总评成绩　70 %

注：此页不作答题纸，请将答案写在答题纸上

一．填空题 (每空 1 分，共 20 分)

1. 嵌入性、专用性与计算机系统是嵌入式系统的　　3 个基本要素。

2. ARM7 微处理器采用　冯.诺依曼　总线架构； ARM9 微处理器采用　哈佛　总线架构。

3. ARM 有三种含义，分别是　公司名称　、一类处理器的通称　与　技术名称　。

4.在 RISC 中, 完成数据在寄存器和外部存储器之间的传输采用独立的　load　与 store 指令。

5. ARM 核有两个指令集，分别是　ARM　、 THUMB　。

6. ARM 公司提供的专门用于　ARM 相关应用开发和调试的综合性软件为：　ADS1.2　。

7.ARM 微处理器的工作状态一般有两种，两种状态之间的切换采用　BX　指令。

8. 当程序状态寄存器的内容为　0xA030 时, 说明处理器运行　THUMB　状态和 用户　模式下。

9. ARM 核中的　R14(或 LR)　寄存器，常用作保存子程序的返回地址。

10.汇编语言的源程序主要由　指令　、 伪指令　、 语句标号　与　注释　等四部分组成。

二．简答题 (56 分，每小题　7 分)

1. 简述嵌入式系统的概念、组成及特点。

答：嵌入式系统是以应用为中心，　以计算机技术为基础，　采用可裁减软硬件，　适用于对功能、可靠性、成本、体积、功耗等有严格要求的专用计算机系统。一般由嵌入式微处理器　　、外围硬件设备、嵌入式操作系统以及用户的应用程序等四个部分组成。其特点有

1)嵌入式系统通常是面向特定应用的

2) 嵌入式系统是将先进的计算机技术、半导体技术和电子技术与各个行业的具体应用相结合后的产物

3)嵌入式系统的硬件和软件都必须高效率地设计，　　量体裁衣、去除冗余

4)嵌入式系统和具体应用有机地结合在一起，它的升级换代也是和具体产品同步进行

5)为了提高执行速度和系统可靠性，　嵌入式系统中的软件一般都固化在存储器芯片或单片机本身中

6)嵌入式系统本身不具有自主开发能力。

2. S3C2440A 中的中断控制器可以从多个中断源接收中断请求，　　简述中断控制器请求的　FIQ 及 IRQ 中断机制。

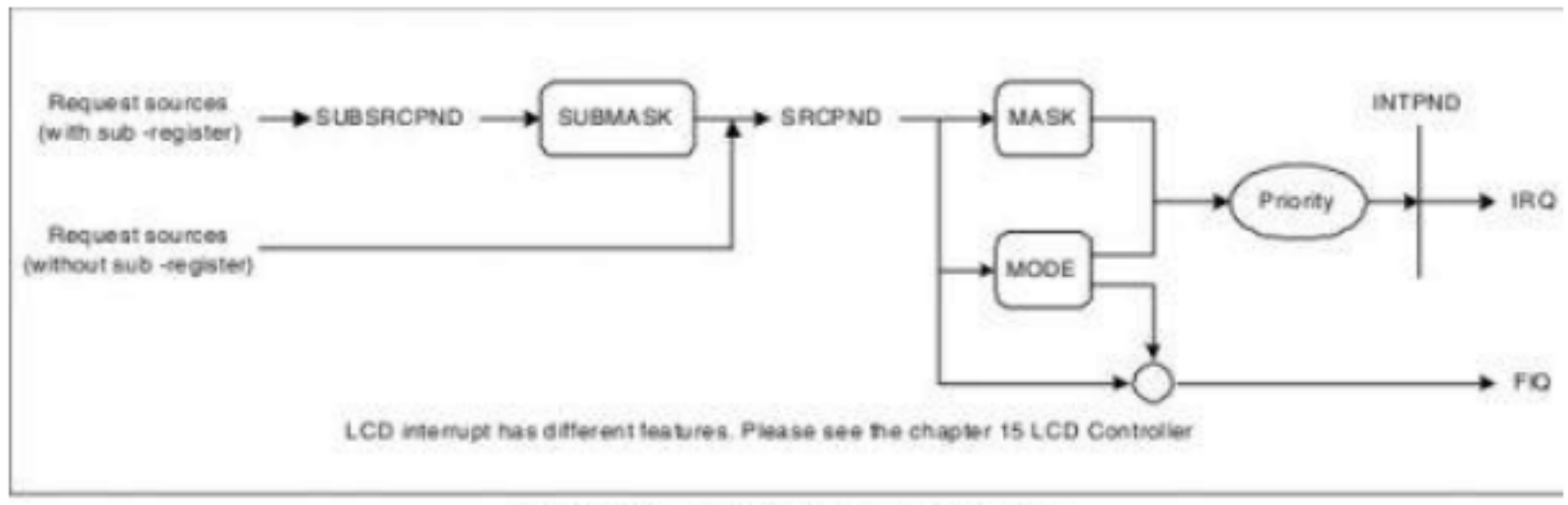


Figure 14-1. Interrupt Process Diagram

3.试说明　看门狗　Watchdog 的作用和原理？

答：在嵌入式应用中，　　CPU　必须可靠工作，即使因为某种原因进入了一个错误状态，系统也应该可以自动恢复。看门狗的用途就是使微控制器在进入错误状态后的一定时间内复位。其工作原理是在系统正常工作时，用户程序每隔一段时间执行喂狗动作　　,如果系统出错，喂狗时隔超过看门狗溢出时间，那么看门狗将会产生复位信号，使系统复位。

4. 简要说明一下　ARM　的异常的响应和返回的过程。

答：

A.　异常的进入：

（1）将下一条指令的地址存入相应连接寄存器　　LR，以便程序在处理异常返回时能从正确的位置重新开始执行。

（2）将 CPSR 复制到相应的　SPSR 中。

（3）根据异常类型，强制设置　　CPSR 的运行模式位。

（4）强制 PC 从相关的异常向量地址取下一条指令执行，从而跳转到相应的异常处理程序。

也可以设置中断禁止位来阻止其他无法处理的异常嵌套。

B.　异常的返回：

（1）将链接寄存器　LR 的值减去相应的偏移量后送到　　PC 中。

（2）将 SPSR 复制回 CPSR 中。

（3）如果进入时设置了中断禁止位，那么清除该标志。

5. 在 C 语言与汇编程序混合编程中，子程序调用的　　ATPCS 规定了哪些基本规则。简要说明寄存器使用规则。

答：基本规则有三个方面内容，　　分别是寄存器的使用规则及其相应的名字，　　数据栈的使用规则，参数传递规则。

A 寄存器的使用规则　：

（1）子程序通过寄存器　　R0~R3 来传递参数。　　这时寄存器可以记作：　　A0~A3　，被调用的子程序在返回前无需恢复寄存器　　R0~R3 的内容。

（2）在子程序中，使用　　R4~R11 来保存局部变量　,这时寄存器　R4~R11 可以记作：　V1~V8 。如果在子程序中使用到　　V1~V8　的某些寄存器，子程序进入时必须保存这些寄存器的值，在返回前必须恢复这些寄存器的值，　　对于子程序中没有用到的寄存器则不必执行这些操作。　　在 THUMB　程序中，通常只能使用寄存器　　R4~R7 来保存局部变量。

（3）寄存器　R12 用作子程序间　scratch 寄存器，　记作　ip；在子程序的连接代码段中经常会有这种使用规则。

（4）寄存器 R13 用作数据栈指针， 记做 SP；在子程序中寄存器 R13 不能用做其他用途。 寄存器 SP 在进入子程序时的值和退出子程序时的值必须相等。
（5）寄存器 R14 用作连接寄存器， 记作 lr ；它用于保存子程序的返回地址， 如果在子程序中保存了返回地址，则 R14 可用作其它的用途。
（6）寄存器 R15 是程序计数器，记作 PC ；它不能用作其他用途。
（7） ATPCS 中的各寄存器在 ARM 编译器和汇编器中都是预定义的。

6.简述 ARM 处理器的寻址方式，并回答在 ATPCS 规则中，规定数据栈采用那种类型。
答： ARM 指令系统支持以下 7 种寻址方式：
（1） 寄存器寻址；
（2） 立即寻址；
（3） 寄存器间接寻址；
（4） 变址寻址；
（5） 寄存器移位寻址；
（6） 多寄存器寻址；
（7） 堆栈寻址。

在 ATPCS 规则中，规定数据栈采用满递减类型。

7. 如果 S3C2440A 的 UART 时钟为 40MHz，要将 UART0 设为正常工作模式 ，并且为偶校验，停止位 1 位，数据位 7 位，波特率为 230400bps，该怎么设置， UART0 的线控制寄存器为 ULCON0，波特率设置寄存器为 UBRDIV0 。

答： UBRDIV0=0X09;
ULCON0&=0XFFFFFF00;
ULCON0|=0X2A;
三 程序设计（ 24 分，每小题 12 分）
1、通过控制 GPF0,GPF1,GPF2,GPF3 的控制实现对 LED1,LED2,LED3,LED4 轮流亮灭的控制。 LED 灯通过电源与限流电阻与 ARM 的 I/O 口相连，当 I/O 口为低电平时，点亮 LED.要求将程序缺失的部分补全。 （12 分）

```
AREA |DATA|,CODE,READONL  Y
    ENTRY
    ldr r13,=0x1000
    IMPORT LedMain
    b LedMain
    END

#define GPFCON (*(volatile unsigned *)0x56000050)
#define GPFDAT (*(volatile unsigned *)0x56000054)
#define GPFUP (*(volatile unsigned *)0x56000058)
int LedMain()
{

    void Delay(unsigned int);
    unsigned char ledtab[]={0xfe,0xfd,0xfb,0xf7};
```

```
    int i;
    GPFUP&= （0XFFFFFFF0）;            //使能上拉   F3-6
    GPFCON&= （0XFF00）;
    GPFCON|=（0X0055;）                  //将 F3-6 设为输出口
    while(1)
    {
        for(i=0;i<4;i++)
        {
            （GPFDAT=ledtab[i];）
                Delay(70);
            }
        }
        return(0);

}


void Delay(unsigned int x )
{
    unsigned int i,j,k;
    for(i=0;i<=x;i++)
        for(j=0;j<=0xff;j++)
            for(k=0;k<=0xff;k++);

}
```

2、EINT2 接 S3C2440A 的 GPF2，并接上拉电阻，下降沿触发中断，试将主程序和中断服务程序缺失的部分补全。 (12 分)

```
int  Main （）
{
Eint_Init();
Enable_Eint()
While(1)
{
Uart_Printf(  “main is running  ”);

}
void  Eint_Init(void)
{
rGPFCON= rGPFCON &~(3<<4)|(1<<5);

rGPFUP|=(1<<2);
```

```
rEXTINT0=( rEXTINT0&~(7<<8))|(2<<8);
pISR_EINT2=(unsigned)Eint2_ISR;
}
void    Enable_Eint(void)

{

rEINTPEND=0xffffff;
rEINTMASK=~(1<<19);

rSRCPND|=BIT_EINT2;

rINTPND|= BIT_EINT2;  ;
rINTMSK=~( BIT_EINT2);
}

static    void__irq     Eint2_ISR(void)

{

ClearPending(BIT_EINT2);
Uart_Printf(  “eint2 is running  );”
}
```